ADR-009 — Mensageria (RabbitMQ com Transactional Outbox)

Status: Proposto | Data: 20/10/2026 | Autor(es): Equipe ___

Requisitos/restrições relacionados: REQ02, REQ07, REQ10, REQ11 (emails); REQ14–REQ15 (relatório e ZIP); ADR-001; ADR-002 (hash); ADR-003 (outbox, retentativa, DLQ, idempotência); ADR-005 (pós-processamento assíncrono); ADR-007 (sessão interativa fora da fila); ADR-011; ADR-013.

Contexto

As ADRs 001, 003 e 005 exigem mensageria durável com retentativa exponencial, DLQ e consumers idempotentes. O escopo da fila é o pós-processamento (hash de verificação, relatório, ZIP) e os emails; a sessão de captura é síncrona e usa WebSocket diretamente (ADR-007). O volume é moderado e a equipe pequena: complexidade operacional pesa mais que vazão extrema.

Decisão

Adotar RabbitMQ como broker, com as seguintes regras:

- **Publicação via transactional outbox:** cada serviço grava a mensagem na tabela outbox na mesma transação da mudança de estado; um relay publica no RabbitMQ com publisher confirms e marca a mensagem como enviada.
- **Durabilidade na v1 (nó único):** filas clássicas duráveis e mensagens persistentes; quorum queues quando houver cluster de três nós.
- **Retentativa exponencial:** filas de espera por nível (1 min, 5 min, 30 min, 2 h) com TTL, cujo dead-letter retorna à fila de trabalho; esgotados os níveis, a mensagem vai para a DLQ de tratamento operacional, com alerta (ADR-013).
- **Consumo:** acks manuais e chave de idempotência por mensagem (id do registro e tipo de operação) verificada no banco antes de executar.

Alternativas Consideradas

1. Redis + BullMQ — rejeitada: simples no Node, mas durabilidade dependente da configuração de persistência do Redis, conflitando com a ADR-003.
2. Apache Kafka — rejeitada na v1: operação de cluster, partições e offsets desproporcional ao volume e à equipe.
3. AWS SQS — rejeitada: acopla a v1 a um provedor de nuvem; o deploy previsto é local/Docker (ADR-012).
4. Fila no próprio PostgreSQL (SKIP LOCKED) — rejeitada como fila de trabalho: misturaria carga de processamento com o livro-razão. A tabela outbox, pequena e transitória, é usada apenas para publicação.
5. Plugin delayed-message exchange para a retentativa — rejeitado: plugin externo ao núcleo, com armazenamento não replicado; filas de espera com TTL usam apenas recursos nativos.

Justificativa

- O outbox elimina a escrita dupla banco/fila, garantindo que nenhuma mensagem de operação crítica se perca entre o commit e a publicação (ADR-003).
- Retentativa por níveis e DLQ são implementadas por configuração (TTL e dead-letter), com recursos nativos do broker.
- Filas clássicas duráveis dão durabilidade em nó único; quorum queues só agregam disponibilidade com três nós e ficam para a evolução.
- Operação simples em Docker, management UI e protocolo aberto (AMQP 0-9-1) adequados à equipe.

Cenários de Qualidade (medida de resposta)

- Estímulo: serviço cai logo após o commit de um registro concluído. Resposta: o relay publica a mensagem pendente ao reiniciar. Medida: zero mensagens perdidas; publicação em até 1 minuto após o retorno.

Consequências

Positivas: operações críticas duráveis e reprocessáveis; desacoplamento entre Captura/Registro, Mídia e Notificação; observabilidade nativa.

Negativas: mais um componente de infraestrutura e um relay por serviço; entrega at-least-once (mensagens duplicadas possíveis); nó único do broker sem alta disponibilidade na v1.

Mitigações: idempotência em todos os consumers; alertas de DLQ, profundidade de fila e idade do outbox (ADR-013); o outbox retém as mensagens enquanto o broker estiver indisponível.

Alterações em relação à versão anterior

- Adicionado transactional outbox; escopo limitado ao pós-processamento e emails.
- Quorum queues substituídas por filas clássicas duráveis na v1 (nó único).
- Detalhada a implementação da retentativa exponencial com recursos nativos.